Task A4 – What Should Be Serialized? (4 Marks, K4)

Part A asks for design, pseudocode and reasoning, so the answer is in pseudocode and tables. The Java in the *Alternative* sections is only for illustration. Items marked `<fill>` need your own observations.

A4.1 – Decide what to persist

Field	`transient`?	Reason
`Resident.name`, `Resident.regNo`	No	the data that must survive
`Resident.password`	**Yes**	secret: a serialized file is easy to read, copy and send; passwords must never be stored in plain form, and should be re-entered or kept only as a salted hash elsewhere
`Resident.connection` (live `Connection`)	**Yes**	a database connection is not `Serializable` (without `transient`, `writeObject` throws `NotSerializableException`), and a restored socket or connection would be meaningless anyway because the real connection no longer exists; it is reopened after loading
`Hostel.rooms`, `Room.residents` (lists)	No	they are part of the state; `ArrayList` is `Serializable`

What happens to the object graph when the `Hostel` is serialized:
`writeObject(hostel)` writes the `Hostel`, then follows every reference that is not `transient` or `static`: the `rooms` list → each `Room` → its `residents` list → each `Resident`. The *entire reachable graph* is saved in one stream, so every class in it must implement `Serializable`. If the same object is referenced from two places, it is written once and the sharing is preserved when loaded (back-references in the stream). `static` fields are not saved. On reading, the constructor of a `Serializable` class is not run; fields are restored from the stream, and `transient` fields get their default values (`null`, `0`, `false`).

Run result (sandbox, `HostelDemo`): `Block A room 101: Arun 192311001 password=null con=null`, so the name and register number were restored, while the password and connection were not.

```
import java.io.*;
import java.sql.Connection;
import java.util.*;

class Resident implements Serializable {
    private static final long serialVersionUID = 1L;       // fixed
version number
    String name, regNo;
    transient String password;        // secret: must never be written
to a file
    transient Connection con;         // live resource: not
serializable and meaningless after reload
    Resident(String n, String r, String p) { name = n; regNo = r;
password = p; }
}
class Room implements Serializable {
    private static final long serialVersionUID = 1L;
    int number; List<Resident> residents = new ArrayList<>();
    Room(int n) { number = n; }
}
class Hostel implements Serializable {
    private static final long serialVersionUID = 1L;
    String name; List<Room> rooms = new ArrayList<>();
    Hostel(String n) { name = n; }
}
public class HostelDemo {
    public static void main(String[] a) throws Exception {
        Hostel h = new Hostel("Block A");
        Room r = new Room(101); r.residents.add(new Resident("Arun",
"192311001", "secret123"));
        h.rooms.add(r);
        File f = File.createTempFile("hostel", ".ser");
        try (ObjectOutputStream o = new ObjectOutputStream(new
FileOutputStream(f))) { o.writeObject(h); }  // whole graph
        try (ObjectInputStream i = new ObjectInputStream(new
FileInputStream(f))) {
            Hostel h2 = (Hostel) i.readObject();
            Resident x = h2.rooms.get(0).residents.get(0);
            System.out.println(h2.name + " room " +
h2.rooms.get(0).number + ": " + x.name + " " + x.regNo
                    + " password=" + x.password + " con=" + x.con);
        }
        f.delete();
    }
}
```

A4.2 – Plan for change (phone number added later)

Old files were written by the old class (no `phone`). Reading them with the new class depends on **`serialVersionUID`**:

Situation	Result	Evidence
Class declares `private static final long serialVersionUID = 1L;` in both versions	The old file loads successfully; the new field `phone` gets its default value **`null`**	sandbox run: `Arun phone=null`
No `serialVersionUID` declared	The JVM computes one from the class structure (fields, methods…). Adding `phone` changes it, so loading the old file fails with **`InvalidClassException`**	sandbox run: `java.io.InvalidClassException: Resident; local class incompatible: stream classdesc serialVersionUID = -1485896733400315031, local class serialVersionUID = -4704594282173425439`

So: always declare `serialVersionUID` explicitly. *Adding* a field is compatible and the field defaults to null/0 (the program should handle that). *Removing* a field is also compatible (the data is ignored). *Changing a field's type* or the class hierarchy is incompatible; in that case change the UID deliberately and write a migration, or provide custom `readObject` logic.

Alternative concept and complexity

Store the data as CSV or JSON instead of Java serialization:

```
CSV :  Block A,101,Arun,192311001            (one row per resident)
JSON:  {"name":"Block A","rooms":[{"number":101,"residents":
[{"name":"Arun","regNo":"192311001"}]}]}
```

Aspect	Java serialization	CSV	JSON
File size for n residents	O(n); includes class descriptors once, plus per-object overhead and type tags	O(n); smallest per record, since no field names are stored	O(n); larger per record because every field name is repeated
Time to save / load	O(n); binary, no parsing but uses reflection	O(n); simple parsing	O(n); parsing with a library, usually slowest

Aspect	Java serialization	CSV	JSON
Readable and portable	No; Java only	Yes, any tool	Yes, any language
Safety	Loading untrusted files is a known security risk; secrets must be `transient`	only the columns you choose are written, so a password is never saved by accident	same as CSV
New field added later	needs `serialVersionUID` discipline; old files give `null`	old rows are shorter, so the reader must tolerate a missing last column	old files simply lack the key; readers treat it as missing, which is the most tolerant

All three are O(n) in time and space; the differences are in constants, safety and evolution. Record your own measurement for a given n (e.g. 100,000 residents): file size in bytes and save/load times `<fill>`.